Lecture 2 - Dart OOP I

Classes, Objects, Inheritance, Encapsulation, Polymorphism

Lab: Student Classes and Hierarchy

Fettah KIRAN

Department of AI and Data Engineering

Yıldız Technical University

fkiran@yildiz.edu.tr

Overview

1. Recap Quiz from Week 1
2. Dart Setup: DartPad, SDK, VS Code
3. Variables & Null Safety Essentials
4. Classes, Objects & Constructors
5. Fields, Methods, Getters/Setters, Static
6. Inheritance: extends, super, @override
7. Polymorphism, Abstract Preview, Implements
8. Lab: Student → GraduateStudent, BankAccount, Animal polymorphism

Why Dart for Flutter?

- **One language** for UI + logic: no JS bridge, compiles to ARM/x64 native
- **OOP + modern:** classes, mixins, generics, null safety, async built-in
- **Hot Reload friendly:** JIT in dev, AOT in release — same code
- **App friendly:** classes, inheritance, and method overriding organize app data and behavior

Dart Setup (Zero Excuses)

- **Option A — DartPad:** https://dartpad.dev — runs in browser, zero install, all lab code runs here
- **Option B — Local:** `dart --version` (ships with Flutter SDK) → Dart 3.x
- **VS Code:** Flutter + Dart extensions → `.dart` files get run/debug/lint free
- Check now: `flutter doctor` clean? `dart --version` prints?

Ex 1 — Variables & Null Safety (Run in DartPad)

```dart
void main() {
  var name = 'MTM4681';        // inferred String, non-nullable
  int? score;                  // nullable: currently null
  late String team;            // promised to init later
  team = 'OOPSquad';
  print('$name | $team | ${score ?? 0}');
  // score!  -> crash if null! Use ?? or if-check instead.
  const pi = 3.14159;          // compile-time const
  final now = DateTime.now(); // run-time final
}
```

- `?` = may be null, `!` = trust me non-null (dangerous), `late` = init later
- `required` named params — next slides in constructors

Null Safety in 60 Seconds

- **Non-nullable by default:** `int x;` must be assigned before use
- `int? y` can hold null → check with `y == null`, `y ?? 0`, `y?.abs()`
- `late` delays init — useful for fields set in `initState()` later
- Rule of thumb: never use `!` unless you just null-checked on the line above

Ex 2 — First Class: Student

```
class Student {
  String name;
  int age;
  Student(this.name, this.age);
  void displayInfo() {
    print('Student: $name, Age: $age');
  }
}
void main() {
  var student = Student('Ali', 20);
  student.displayInfo();                    // Student: Ali, Age: 20
}
```

- A class groups related data (fields) and behavior (methods)
- `this.name` shorthand assigns constructor parameters to fields

Constructors: Generative + Named

```
class Student {
  final String name;
  final int age;
  Student(this.name, this.age);
  Student.firstYear(String name) : this(name, 18); // redirecting constructor
}
void main() {
  print(Student.firstYear('Ada').age);   // 18
}
```

- Named constructors clarify intent: `.firstYear()`, `.fromJson()`
- Initializer list (`:`) runs before body — required for `final` fields

Initializer List + const

```dart
class Student {
  final String name;
  final int age;
  const Student(this.name, this.age);
}
void main() {
  const a = Student('Ada', 20);
  const b = Student('Ada', 20);
  print(identical(a, b));                 // true: canonicalized
}
```

- `final` = set once; `const` = compile-time constant, canonicalized
- Use immutable objects when callers should not change their state after creation

Factory Constructors

```dart
class Student {
  final String name;
  final int age;
  const Student(this.name, this.age);
  factory Student.fromMap(Map<String, Object?> data) {
    return Student(data['name'] as String, data['age'] as int);
  }
}
void main() {
  final student = Student.fromMap({'name': 'Ada', 'age': 20});
  print('${student.name}, ${student.age}');
}
```

- `factory` can return cached instance, subclass, or parse logic
- A factory can validate or convert external data before creating an object

Ex 3 — Getters, Setters, Methods

```
class BankAccount {
  double _balance;
  BankAccount(this._balance);
  double get balance => _balance;
  void deposit(double amount) {
    if (amount > 0) _balance += amount;
  }
}
void main() {
  final account = BankAccount(1000);
  account.deposit(500);
  print(account.balance);                    // 1500.0
}
```

- Getter exposes a controlled view of private state; methods validate changes

Static Members

- `static` belongs to the **class**, not the instance — e.g. `Student.currentYear`
- Use for: constants, helpers, factories, and caches
- Cannot use `this` inside static — no instance yet
- A class-level member is shared; instance fields belong to each individual object

Ex 4 — Inheritance: GraduateStudent extends Person

```
class Person {
  String name;
  int age;
  Person(this.name, this.age);
}
class GraduateStudent extends Person {
  String major;
  double gpa;
  GraduateStudent(String name, int age, this.major, this.gpa)
      : super(name, age);
}
void main() {
  final grad = GraduateStudent('Zeynep', 25, 'Computer Eng.', 3.75);
  print('${grad.name}, ${grad.major}');  // inherited name + own major
}
```

- `extends` = IS-A; `super(name, age)` initializes the parent portion
- `@override` catches signature typos at analyze time

super & @override Rules

- `super(...)` or `super.x` must init parent before child body runs
- `@override` marks intentional replacements, such as `introduce()` or `makeSound()`
- Parent private (`_field`) is library-private — child in same file sees it, other files don't
- Subclasses should preserve the promises made by their parent class

Ex 5 — Polymorphism via Subtype

```
  void makeSound() {
    print('Some generic animal sound');
  }
}
class Dog extends Animal {
  @override
  void makeSound() => print('Woof!');
}
class Cat extends Animal {
  @override
  void makeSound() => print('Meow!');
}
void main() {
  final List<Animal> animals = [Dog(), Cat()];
  for (final animal in animals) animal.makeSound();
  // Woof!
  // Meow!
}
```

- Same call `makeSound()`, different behavior — **polymorphism**
- A `List<Animal>` can hold different subclasses and dispatch each override

Abstract Class Preview

- `abstract class Shape { double area(); }` — no instances, subclasses must implement
- Week 3 deep-dive: `abstract class Solver { ... }` + Newton vs Bisection
- Today: understand concrete base classes and overrides; abstract classes are an extension
- Abstract ≠ interface: abstract can hold state + concrete helpers

Interface via implements

- Dart has no `interface` keyword — every class is an interface
- `class Circle implements Shape` must re-implement **everything** (no inherited code)
- `extends` = reuse code; `implements` = guarantee API; `with` (Week 3) = mixin reuse
- Prefer `extends` when you need inherited implementation; use `implements` to promise an API without inheriting code

Ex 6 — Cascades (..)

```dart
class Student {
  final String name;
  final int age;
  Student(this.name, this.age);
  void displayInfo() => print('Student: $name, Age: $age');
}
void main() {
  final student = Student('Ali', 20)..displayInfo();
}
```

- `..` chains calls on the **same object** — replaces verbose temp vars
- Flutter twin: `Paint()..color = red..strokeWidth = 2` (Week 10)

Enums (Enhanced)

- `enum Op { add, mul, remove }` — finite choices for application actions
- Enhanced enums hold fields/methods: `enum Norm { l1('Manhattan'), l2('Euclid'); ... }`
- Use in `switch` — analyzer warns if you miss a case
- Enums are a useful way to represent a fixed set of states or choices

Null Safety in OOP

```dart
class Student {
  final String name;
  String? email;                          // optional -> nullable
  late final String id;                   // set in register()
  Student({required this.name, this.email});
  void register(String newId) => id = newId;
}
void main() {
  var s = Student(name: 'Ada');           // email null OK
  s.register('YTU-001');
  print(s.email ?? 'no email');           // ?? fallback
}
```

- `required` named params kill null bugs at call site
- `late final` for DI / `initState` patterns in Flutter

Ex 7 — toString / == / hashCode

```
class Student {
  final String name;
  final int id;
  const Student(this.name, this.id);
  @override String toString() => 'Student($name, $id)';
  @override bool operator ==(Object o) =>
      o is Student && o.name == name && o.id == id;
  @override int get hashCode => Object.hash(name, id);
}
void main() {
  const a = Student('Ada', 1);
  const b = Student('Ada', 1);
  print(a == b);                          // true
}
```

- Always override all three together — `Set` / `Map` / `==` in tests depend on it
- Not required in today's lab; useful when objects need value-based equality

Packages & Imports

- `import 'dart:math';` → `sqrt`, `pi`, `Random` — core SDK
- `import 'package:my_app/student.dart';` → your own files in a Dart package
- Relative: `import 'student.dart';` inside the same folder/lib
- `as`, `show`, `hide`: `import 'dart:math' as m show sqrt;` avoids name clashes

dart format & analyze

- `dart format .` — canonical style, required before Final submission
- `dart analyze` — catches missing `@override`, dead null-checks, unused imports
- VS Code: format-on-save ON; problems panel = analyzer live
- CI habit: format + analyze clean = professionalism points on project rubric

Ex 8 — Polymorphism: Student Hierarchy

```
  String name;
  int age;
  Person(this.name, this.age);
  void introduce() => print("I'm $name, $age years old");
}
class Student extends Person {
  String studentId;
  double gpa;
  Student(String name, int age, this.studentId, this.gpa)
      : super(name, age);
  @override
  void introduce() => print('Student: $name, ID: $studentId, GPA: $gpa');
}
class GraduateStudent extends Student {
  String thesisTopic;
  GraduateStudent(String name, int age, String id, double gpa, this.thesisTopic)
      : super(name, age, id, gpa);
  @override
  void introduce() => print('Graduate: $name, Topic: $thesisTopic');
}
void main() {
  final List<Person> people = [
    Student('Ahmet', 20, 'S123456', 3.5),
    GraduateStudent('Mehmet', 25, 'G789012', 3.8, 'Machine Learning'),
    Person('Ayse', 30),
  ];
  for (final person in people) {
    person.introduce();
  }
}
```

- One `List<Person>`, several runtime types, one `introduce()` call
- The runtime object determines which overridden method runs

Common Errors (Save 30 min)

- `const` needs all-final fields + const super — else "Can't define const constructor"
- `!` on nullable after async gap — value may turn null; use local copy + null check
- Missing `@override` can hide a misspelled method instead of overriding it
- `==` without `hashCode` → `Set` dedup fails in tests
- `_name` is library-private in Dart; it is not a protected member visible to subclasses in other libraries

Quick Check ✋

- **T1:** What does `_balance` communicate in `BankAccount`? Can another Dart library access it?
- **T2:** Where must the initializer list go — before or after the constructor body? What can it init that body cannot?
- **T3:** A `Person` variable refers to a `GraduateStudent`. Which `introduce()` runs? What OOP term is this?

- Answers: (1) library-private; other libraries cannot access it (2) before body (`:`), initializes fields/superclass (3) `GraduateStudent.introduce()` — polymorphism/dynamic dispatch

Dart vs Java / Python (For Seniors)

	Dart	Java	Python
Null safety	Sound (`?` , `!` , `required`)	Optional/ `@Nullable`	None (None everywhere)
Constructors	Generative/named/factory/const	Overloaded only	`__init__` only
Inheritance	Single + `implements` / `with`	Single + interfaces	Multiple
Encapsulation	Library-private (`_name`)	Access modifiers	Convention (`_name`)
Best for course	Native Flutter compile	Verbose for mobile	Slow on phones

- Focus on Dart idioms: `..` , `?.` , `??` , `const` , named constructors

Mixin Teaser → Week 3

- `mixin Printable { String dump(); }` then `class Student with Printable`
- Mixins = reuse without inheritance tree — Week 3: `Validatable`, `Serializable`
- Today: master classes, encapsulation, `extends`, and `@override`; Week 3 adds `with` + `Future<Solution>`

Predict the Output 🧠

```dart
class Person {
  void introduce() => print('Person');
}
class Student extends Person {
  @override
  void introduce() => print('Student');
}
void main() {
  Person person = Student();
  person.introduce();
}
```

- What prints, and why does a `Person` variable run the `Student` implementation? (Answer: `Student` — runtime dispatch uses the object's actual type.)

Lab — OOP: Student Classes and Hierarchy

60 min: create classes, encapsulate state, extend, override, demo

Lab Exercise 1 — Student Class (10 min)

1. Create `01_student.dart` with `name`, `age`, a constructor, and `displayInfo()`.
2. Create `Student('Ali', 20)` and call `displayInfo()`.
3. Run with `dart run 01_student.dart`.

> - **Done when:** the program prints the student's name and age.

Lab Exercise 2 — Inheritance (15 min)

1. Create `Person` with `name` and `age`.
2. Create `GraduateStudent extends Person` with `major` and `gpa`.
3. Initialize inherited fields with `super(name, age)`.
4. Run `02_graduate_student.dart` and display graduate-student information.

Lab Exercise 3 — Encapsulation (10 min)

1. Create `BankAccount` with private `_accountNumber` and `_balance`.
2. Add `deposit(amount)` that accepts only positive amounts.
3. Expose the balance through a getter; do not expose a setter.
4. Run `03_encapsulation.dart` and check the balance before and after depositing.

Lab Exercise 4 — Polymorphism (10 min)

1. Create `Animal.makeSound()`, then override it in `Dog` and `Cat`.
2. Store both objects in variables typed as `Animal`.
3. Call `makeSound()` on each; observe the runtime-specific output.

Lab Exercise 5 — Student Hierarchy Challenge (15 min)

1. Build `Person` → `Student` → `GraduateStudent`.
2. Give each class an `introduce()` method; override it in each subclass.
3. Put a `Student`, `GraduateStudent`, and `Person` in a `List<Person>`.
4. Loop through the list and call `introduce()` to demonstrate polymorphism.

Extension Challenge 🚀

- Add input validation for `age`, GPA, and account deposits
- Add an optional `email` field to `Student` (`String?`)
- Override `toString()` in the student hierarchy
- Add subclasses to `List<Person>` and explain dynamic dispatch

Homework 📝

- Finish the Student hierarchy challenge and keep the Dart files for reference
- Run `flutter doctor -v` — must be green (except optional desktop/Xcode) by Week 3
- Read: Dart tour (https://dart.dev/language) — classes + enums sections
- Form project team (2 to 3 person) — list + 2 app ideas due Week 4

Practice Resources

- **DartPad** — experiment with the class examples in a browser
- **Dart Language Tour** — classes, constructors, enums, null safety
- **Dart Language Tour: Classes** — constructors, inheritance, and methods
- **Flutter Codelab** — re-run the Counter app

Debugging Tips

- Red squiggle? Hover → read analyzer message → `dart analyze` in terminal
- `print(object.runtimeType)` reveals which subclass was created
- Breakpoint in VS Code inside `introduce()` or `makeSound()` to observe dispatch
- Check constructor arguments and the `super(...)` call when inherited fields are missing

Next Week Preview

- **Week 3: Dart OOP II — Abstraction, Encapsulation, Mixins, Async**
- `abstract class Solver`, generics, `Future` — Lab: numerical method class (Bisection/Newton)
- Bring: completed Week 2 Dart files + `flutter doctor` clean
- Pre-read: `Future` vs `Stream` (one page, Dart tour async)

Questions & Discussion

Thank you!

Install check: `flutter doctor` *+ practice on given HW — see you Thursday 9:00!*